Project: Secure Communication

Practical Cryptography for Data Exchanges

Zoi Matsantoni

Project Objectives

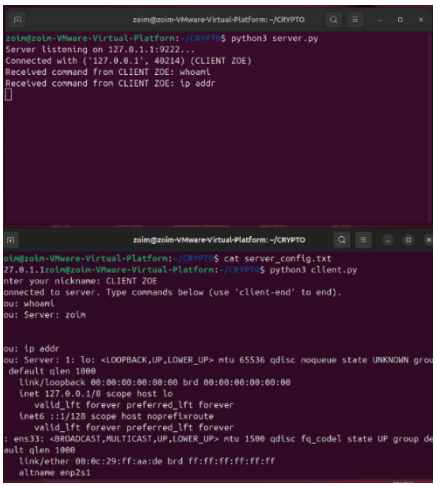
The goal of this project was to develop a secure communication system between a client and server using sockets through a progressive, four-phase approach. Each phase builds on the previous one to create a comprehensive security solution that addresses key aspects of secure communication. The project also required the use of Wireshark to monitor network traffic and identify any confidentiality issues during data exchanges.

Problem statement and the importance of securing communications

This project highlights the vulnerabilities of plaintext communication by using Wireshark to monitor unencrypted data exchanges. This is a significant threat in command and control applications, where unauthorised access could allow malicious parties to execute harmful commands or access sensitive information (ie, man-in-the-middle attacks). To address these risks, secure communication mechanisms such as encryption, authentication, integrity checking, and non-repudiation have been implemented in this project. By phase 4 of this project, I ensure that data transferred between the client and the server cannot be read by external, unauthorised parties and that communicating parties can verify each other's identities.

Server and client functionality

The server and client functionality in this project revolves around creating a secure communication channel via socket programming. The server listens for incoming client connections, executes commands sent by the client, and returns the results, whilst the client connects to the server (via IP found in config file), sends Linux commands (eg. In screenshot: whoami) and displays the output received by the server in the client terminal (eg. In screenshot: zoim). To manage multiple clients and shared resources, both the server and the client use mutex locks to synchronise access, avoiding conflicts in the system.



Directions for running the different Scripts

Phase 1

At this stage, both the server and client scripts are in the same directory. The server dynamically retrieves its own IP address, so it is not passed as a parameter yet. Later, I separate them into different directories with a shared folder for configuration files and keys.

Run command in this format: <python3> <server.py OR client.py>

Phase 2

Implemented os.getcwd() in both scripts, allowing them to be placed in the same directory without hardcoded paths. The code now generates additional files: one for the client, one for the server, and a shared location for .pem key files and for config file (to retrieve IP address). The client also now supports a help command.

Again, run first command in this format: <python3> <server.py OR client.py>

Phase 3

Same usage as before.

Phase 4

The client script now requires the server's IP address to be passed as a parameter. The help command has also been updated to include an option for displaying certificate information.

Run command in this format to launch server script: <python3> <server.py> <server_IP>

Run command in this format to launch client script: <python3> <client.py>

If IP address isn't passed as a parameter, a prompt will be displayed in the server terminal to show how to properly launch the script.

Overview of the asymmetric and hybrid encryption mechanisms used

Whilst Phase 1 of the project simply establishes an unencrypted communication system between the client and server, as the project develops, more security enhancements are introduced through encryption mechanisms. In Phase 2, asymmetric encryption is used to encrypt data sent between the client and server, as both parties generate RSA key pairs. Asymmetric encryption provides a secure way of exchanging data, with the client and server using their public and private key pairs to encrypt and decrypt messages (as illustrated in the Phase 2 PDF asymmetric encryption diagram). This ensures that data remains confidential and protected from unauthorised access, however the encryption process can be slower compared to other methods, especially for large data exchanges. I will discuss this further during the Wireshark observations later in the report.

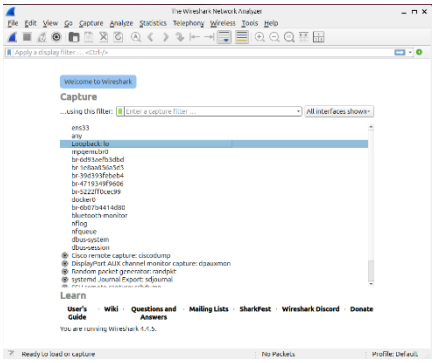
Phase 3 introduces hybrid encryption, which combines RSA for key exchange with AES for data encryption. RSA is employed to securely exchange the AES session key, while AES takes care of the actual data encryption (I demonstrate how this works in greater detail in the Phase 3 hybrid encryption diagram). This hybrid approach provides faster and more efficient communication than relying on RSA alone, and using RSA and AES for data exchange enhances both security and performance, making it particularly well-suited for scenarios where both speed and confidentiality are valued.

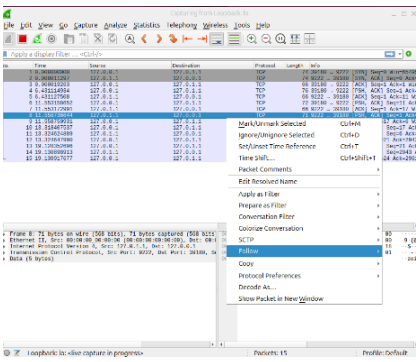
Synchronisation of communication

Initially, I found the requirement to have synchronisation of communication challenging, as I hadn't fully incorporated it into my lab work during class. However, by using mutex locks as prompted in the project outline, the communication between the client and server is now well-managed, ensuring data is exchanged in the correct order without conflicts and the script now smoothly handles communication with multiple clients at once.

Wireshark Observations

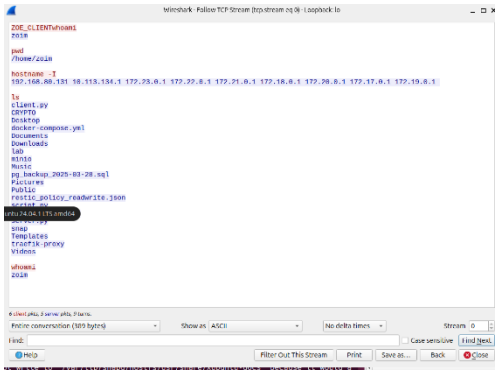
As I mentioned before, Wireshark was used to monitor network traffic and to show how effective the encryption mechanisms were. The screenshots here shows that the traffic channel I monitored was Loopback.io, as the server and client were communicating over the localhost and this network interface is useful for the observation of interactions within the same machine without external network interference (it isolates the network traffic to the machine used).



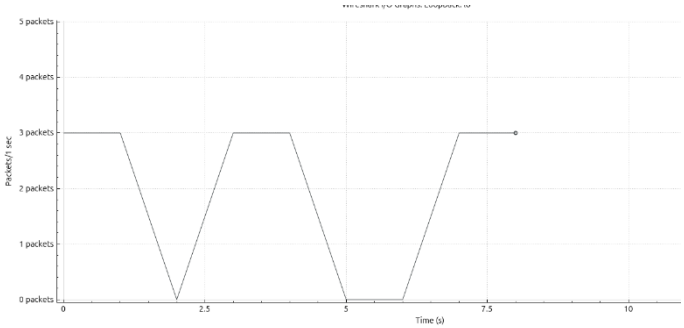


Phase 1 – Plaintext communication

The screenshot here shows traffic during the first phase of the project (plaintext communication). Here, all commands (client commands in red) and responses (server responses in blue) were visible in the packet captures, exposing what could be potentially sensitive information.

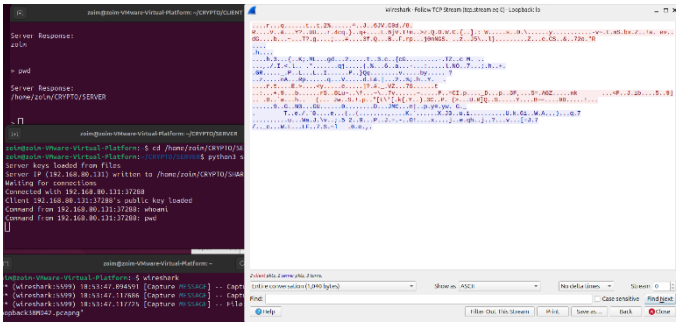


I used Wireshark's I/O graphs to analyse data exchange performance (packet distribution and timing across 6 command transmissions between client and server). When capturing traffic in Phase 1 of the project, the graph shows that for plaintext communication, data was transferred over around 8 seconds. The 3 flat-topped peaks of equal height indicate a fairly steady and consistent transmission.

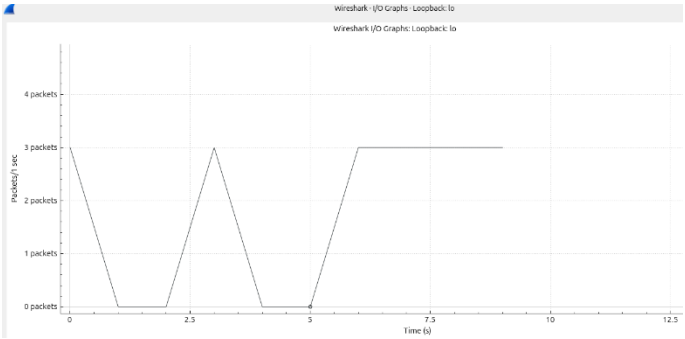


Phase 2 – Asymmetric encryption

Once RSA encryption was implemented, the key exchange process was protected from eavesdropping, as shown in the second Wireshark screenshot here (again, red indicates client commands, blue indicates server response, but this time not in readable format, thus greater security).



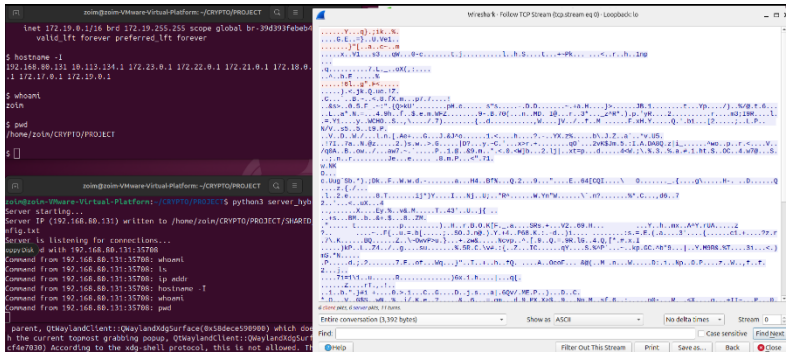
When analysing the performance of Phase 2, we can see that graph extends to around 9 seconds, still with a maximum of 3 packets per second. I looked into why there is a sharp peak followed by a plateau, and came across explanations referring to *overhead*, a term which I was unfamiliar with. After more research, I learnt that the graph suggests that the encryption process causes an initial delay (overhead) at the beginning of communication, but as the transmission continues, the system stabilises. While there were no major



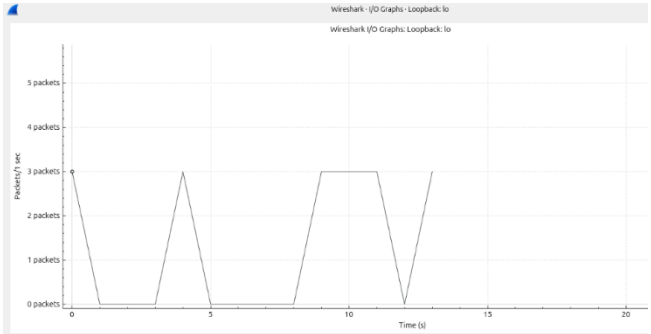
issues/hindrances regarding the performance of adding asymmetric encryption to the communication, as data being exchanged was minimal, this slight delay nonetheless suggests that RSA alone may be inefficient for continuous communication with larger data transfers.

Phase 3 – Hybrid encryption

The Wireshark screenshot here for hybrid encryption approach (RSA & AES) continues to show that traffic is successfully encrypted and meaningful information cannot be deciphered without the encryption keys.



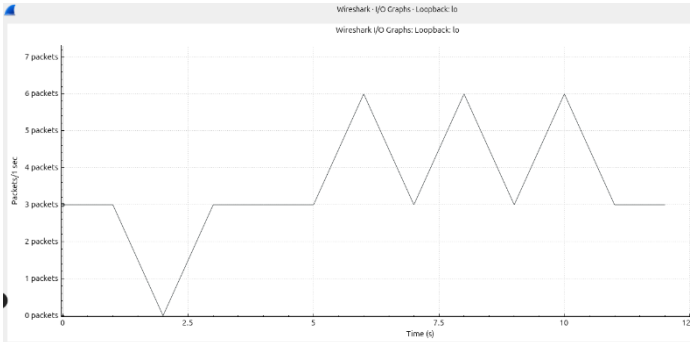
The I/O graph for Phase 3 shows a longer transmission time of roughly 13 seconds for the communication, with wider gaps between activity. These wider gaps suggest that more time is required to establish secure sessions between transmissions, probably because of the time needed for the system to switch between RSA and AES.



I had expected communications for Phase 3 of the project to be evidently more efficient than for Phase 2, which these graphs above do not really prove, so I decided to re-observe performance differences between asymmetric encryption communication and hybrid encryption communication when transferring greater number of data packets (this time sending 15 commands from client to server). After doing so, the differences in performance became a bit clearer (as shown in the screenshots below).

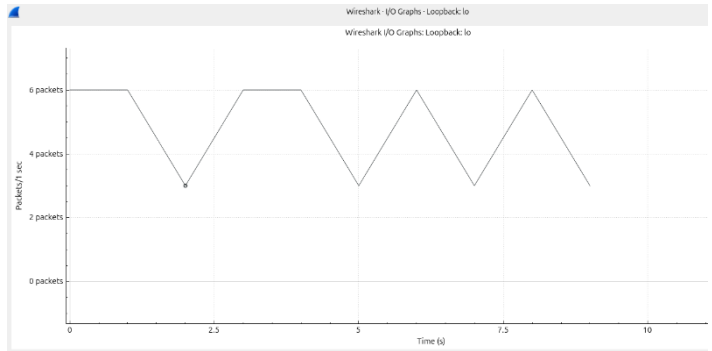
Phase 2 – asymmetric encryption (again)

The first screenshot on the left here shows that the asymmetric encryption approach achieved a transmission rate of 6 packets per second, but with a notable drop to 0, which represents a transmission pause/interval between activity, over the period of 11.6 seconds.



Phase 3 – hybrid encryption (again)

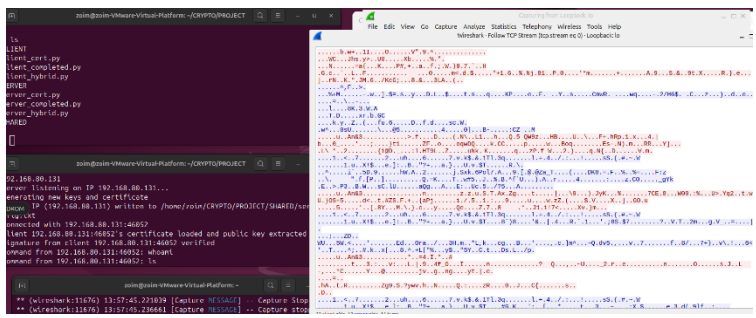
In comparison, the hybrid encryption method also reached 6 packets per second but maintained a minimum rate of 3 packets per second throughout instead of dropping to 0, and it completed the process in around 9 seconds.



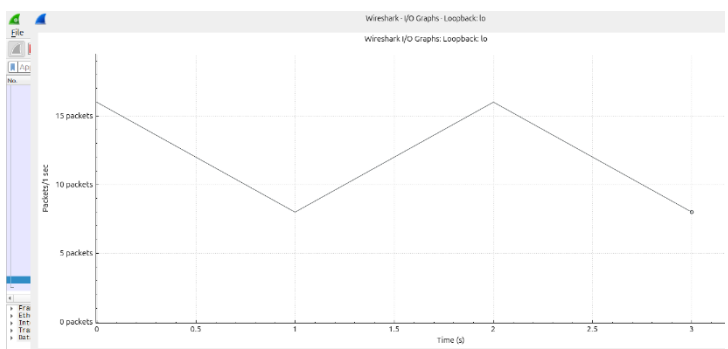
This comparison does in fact support the idea that the hybrid approach ensures more consistent communication without pauses, making it a more efficient alternative to RSA alone, especially when dealing with larger data transfers.

Phase 4 – Hybrid encryption & digital signatures/certificates

As shown in this Wireshark screenshot here, communications between the client and server remain encrypted during Phase 4 of the project, as we are still using hybrid encryption for communication. The addition of digital signatures/certificates during this phase not only strengthens security, but also leads to further performance improvements.



When testing out the performance by sending over 6 commands from the client's terminal (screenshot to the left), the system achieved a transmission rate of 16 packets per second, completing the process in just 3 seconds. When executing fifteen commands, the system maintained the same rate of 16 packets per second for approximately 6 seconds. These results indicate that while certificates introduce some initial overhead (delay), they enable significantly faster transmission once communication is established.



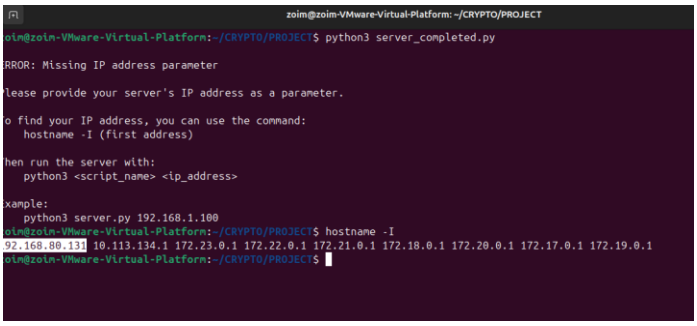
Overall, the Wireshark analysis has confirmed that, as expected, plaintext communications between a client and server are easily observable, whereas implementing encryption enhances security by preventing unauthorised interception. Performance improves further when using hybrid encryption rather than relying solely on asymmetric encryption, and the certificate verification process becomes especially beneficial in longer communication sessions, enhancing overall efficiency despite the added security measures.

Possible Enhancements

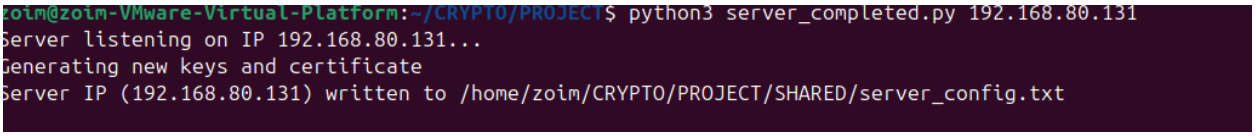
While the final communication system (Phase 4) offers strong security, there are several ways in which the system could be further improved, one of which being the incorporation of Elliptic Curve Cryptography (ECC) as an alternative/complement to RSA, as ECC provides the same level of security as RSA but with smaller key sizes and improved efficiency. Also, the authentication process could be further strengthened by introducing a username and password system to add an extra layer of security beyond certificate verification.

Technical choices and challenges

The scripts were developed in Python, as this was the language used in our class labs, and are designed to work with Linux commands, as I now feel more confident working with this operating system. I worked on a VMWare Workstation Pro virtual machine.

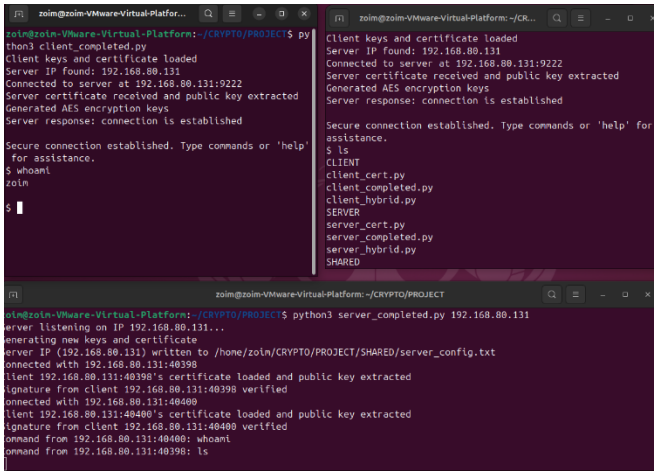


Throughout the development of the scripts, I initially wrote code to determine the server's IP address dynamically and store it in a configuration file for the client to access, rather than hardcoding a specific IP address. However, after reviewing the project requirements, I realised that the IP address needed to be supplied as a script parameter instead. To address this, I made the necessary changes in Phase 4, and now with this adjustment, I am confident that the project fully meets all the specified requirements.



Learning Outcomes

This project provided valuable experience with several technologies and concepts that were previously unfamiliar to me. Before this, I had not used Wireshark for network traffic analysis, but learning to capture and interpret network packets was relatively straightforward and gave me insights into communication security.



As mentioned at the beginning of the report, one of the project requirements was to implement thread synchronisation using mutex locks to manage multiple client connections. However, this was a concept I had not previously worked with. After conducting research and experimenting with mutex locks, I tested the scripts to confirm that multiple clients could connect simultaneously without issues, and it worked as expected, as shown in the screenshot here.

During the development of Phase 4, I faced difficulties optimising the code, which led to "broken pipeline" errors and certificate validation failures. I think that these issues were likely due to improper certificate validation. I was unable to resolve them while keeping the code as concise as I had initially intended, so I reverted to a longer script, which included additional error handling (try-except blocks).

Conclusion

In conclusion, this project offers a solid approach to securing client-server communication while still considering the impact of security on performance. The step-by-step implementation gave me good insights into how different security mechanisms contribute to the overall system.

The final version of the system meets the requirements set out in the project outline, in that the communication system created ensures confidentiality, integrity, authentication, and non-repudiation through digital signatures and X.509 certificates. The hybrid encryption approach balances security needs with performance considerations, making it practical for real-world applications, especially when handling larger data exchanges.

I found the Wireshark analysis particularly interesting, as it helped me visualise and understand the performance differences between asymmetric and hybrid encryption approaches. The I/O graphs clearly showed how each security layer affected communication speed and efficiency.

What I found most beneficial was creating diagrams that illustrated how each phase of the project worked. This exercise really helped cement my understanding of the different encryption mechanisms, reinforced what we covered in class, and gave me a clear picture of the two-way communication system between client and server. Seeing these processes visualised made the abstract concepts much more concrete and memorable.